

금융·증권 IT / 김경우

기능을 만드는 것을 넘어, 시스템이 움직이는 방식을 설계합니다.

복잡한 구조를 더 단순하게 나누고, 문제를 수치로 측정해 개선하며, 아이디어를 실제 운영 환경까지 연결해온 개발자입니다.

SELECTED WORK

01 폴더 생성 자동화 리팩토링

업무 규칙을 Flow에서 데이터로

02 축제 서비스 Cardinal

지도 페이지 성능 최적화

03 베팅 기반 강의실 예약

동시 요청에서의 데이터 정합성

— 금융·증권 IT / 김경우

실제 사용되는 서비스를 이해하고, 더 나은 방향으로 개선합니다.

프론트엔드를 중심으로 서비스의 구조와 사용자 경험을 함께 다뤄온 개발자입니다.

화면을 구현하는 것에서 그치지 않고, 데이터가 어떻게 흐르고 책임이 어느 계층에 있어야 하는지를 고민해왔습니다.

인턴십과 프로젝트를 거치며 업무 자동화 리팩토링, 렌더링 성능 개선, 백엔드 동시성 제어를 경험했습니다.

WAY 01

분리

Separate Concerns

고쳐야 할 지점이 코드 여기저기 흩어지지 않도록, 규칙을 데이터와 제약으로 옮겨 한곳에서 관리합니다.

CRM 인턴

업무 규칙을 마스터 데이터로 옮겨 액션 184 → 41

WAY 02

측정

Measure Bottlenecks

“느리다”는 인상에 머무르지 않고, 수치로 확인하며 효과가 없던 시도까지 기록해 실제 병목을 찾습니다.

Cardinal

지도 페이지 INP 1130ms → 34ms 개선

WAY 03

정합성

Keep Data Correct

요청이 하나씩 올 때만 맞는 코드가 아니라, 동시에 겹쳐도 값이 어긋나지 않는 구조를 만듭니다.

예약 백엔드

비관적 락 · 부분 유니크 인덱스로 동시성 제어

— 경험으로 정리한 기술 영역

사용자 화면부터 내부 시스템 연동과 배포까지,
서비스가 움직이는 과정 안에서 기술을 익히고 적용했습니다.

● Core ● Used

Frontend 화면, 상태 관리, 라우팅, API 응답 흐름 구현	React TypeScript Vite TanStack Query React Router Tailwind CSS Styled-components
API / Integration 프론트엔드와 내부 시스템 사이 연동 계층 구현	Node.js Azure Function HTTP API REST API Bearer Token Auth CORS Apidog
Cloud / Deployment 정적 프론트엔드·서버리스 API 배포와 자동화	Azure Function App Azure Blob Storage GitHub Actions Git / GitHub Storage Static Website Key Vault Managed Identity
Data / Platform 서비스 데이터와 내부 업무 흐름 연결	Dataverse Power Automate SharePoint Supabase MySQL SQL Microsoft Entra ID
Backend / Database 동시 요청에서의 데이터 정합성 처리	Django · DRF PostgreSQL Transaction · Lock pytest DB Constraint
Performance 체감 성능 측정, 렌더링·메인 스레드 병목 개선	INP · Web Vitals Chrome DevTools Performance Main Thread Profiling Rendering Optimization React DevTools Profiler
Also experienced 학습·과제·사이드 프로젝트에서 다뤄본 기술	FastAPI Docker Jenkins Figma

규칙이 바뀔 때마다 Flow를 열어야 했습니다.

01 / 폴더 생성 자동화 리팩토링

Power Automate · Dataverse · SharePoint · CRM 개발 인턴

고객사 업무 자동화 Flow에 폴더 이름과 구조가 그대로 박혀 있었습니다.
업무 규칙을 **Dataverse** 마스터 데이터로 옮기고, 데이터에서 트리를 만들어 반복 생성하도록 바꿨습니다.

01

문제 — 하드코딩된 액션

폴더가 하나 늘 때마다 Flow를 직접 수정해야 했습니다

액션 184 → **41** 개

02

변화 — 수정 지점의 이동

이제 마스터 데이터의 행을 추가하면 폴더가 생깁니다

Flow 내부 → **데이터** 한 곳

— Power Automate RPA

폴더 생성 자동화를 데이터 기반 구조로 리팩토링

BACKGROUND

업무 프로젝트가 상위 프로젝트에 연결되면, SharePoint에 폴더 트리를 만들고 Dataverse에 폴더 메타데이터 레코드를 생성하는 사내 자동화입니다.

자동화가 하는 일

폴더 트리 레벨2·레벨3·하위 폴더 생성

산출물 레코드 Dataverse에 메타데이터 저장

권한 부여 계열사별 폴더 접근 권한 설정

184 → 41actions

약 77.7% 감소

ROLE 단독 수행 · Flow 분석 · 리팩토링

STACK Power Automate · SharePoint · Dataverse

PROBLEM

동작은 했지만, 업무 규칙이 Flow 안에 하드코딩되어 바꾸기 어려운 구조였습니다.

- 1

폴더 하나 = 액션 하나 약 120개

레벨2·레벨3·하위 폴더가 전부 개별 액션 — 폴더 하나만 추가·변경해도 Flow를 직접 수정해야 했습니다.
- 2

분기·반복이 거의 없는 절차형 If 3 · Foreach 2

같은 패턴을 묶지 않고 직렬로 나열 — 액션 수가 폴더 수만큼 무한정 늘어나는 구조였습니다.
- 3

긴 if 식 + 문자열 payload

경로는 긴 if 중첩식, 레코드는 replace 연쇄로 조립 — 이름 하나 바뀌면 줄줄이 깨졌습니다.

LEGACY 184 ACTIONS · 구성

대부분이 단순 폴더 생성 액션

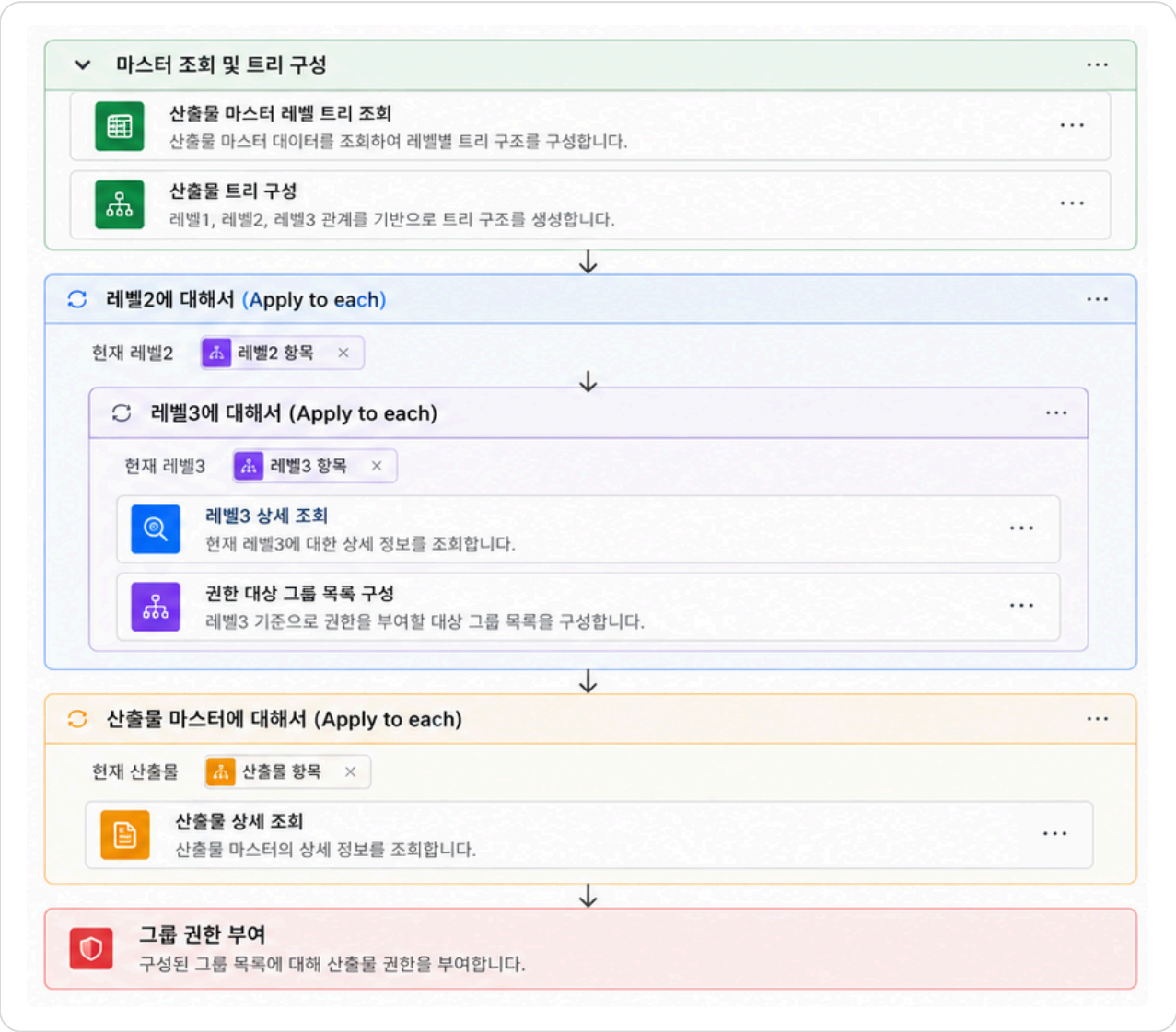


| 그래서 업무 규칙을 Flow에서 **Dataverse** 마스터 데이터로 옮기는 방향으로 리팩토링했습니다.

— Power Automate RPA 리팩토링 · 해결

업무 규칙을 **Dataverse 마스터 데이터로 옮기고,**
데이터에서 트리를 만들어 반복 생성하도록 바꿨습니다.

REFACTORED FLOW · 실제 화면



AFTER 개별 액션 나열이 **그룹화된 반복** 구조로 정돈

핵심 개선

- 1 폴더 구조를 데이터로 관리**
폴더가 바뀌어도 자동화를 고칠 필요 없이 **데이터만 수정**하면 된다.
- 2 중복 폴더 자동 정리**
같은 폴더가 여러 번 나와도 **한 번만** 만들어지도록 데이터를 정리.
- 3 데이터를 안전하게 조립·검증**
문자열 조립을 **구조화된 방식**으로 바꾸고, 쓸 수 없는 문자를 미리 걸러 오류를 방지.

41 actions	데이터 가공 액션이 새로 등장
OpenApiConnection	24
Foreach	4
Compose · 신규	5
Query · Select · 신규	5

핵심은 액션 수가 아니라 수정 지점의 이동
폴더가 바뀌면 Flow가 아니라 마스터 데이터만 고치면 됩니다.

느낌이 아니라 **측정**으로 병목을 찾았습니다.

02 / 학교 축제 서비스 Cardinal

React · TypeScript · Mapbox 기반 축제 지도 페이지 성능 최적화 · 프론트엔드

사용자가 가장 많이 만지는 지도 페이지에서 **리스트를 여닫을 때 프레임이 전부 드랍**했습니다.

가설을 세우고 측정하고, 빗나가면 다시 분석하기를 반복해 **진짜 병목까지 내려간** 경험입니다.

담당 페이지

지도

부스

굿즈

GNB

백오피스

— 학교 축제 지도 페이지

마커·리스트가 열릴 때 프레임이 무너지는 지도

BACKGROUND

동아리에서 맡은 학교 축제 웹 서비스. Mapbox 지도 위에 9개 카테고리 마커와 부스 리스트를 엮었습니다.

지도 페이지가 하는 일

카테고리	선택 시 해당 마커만 지도에 표시
마커 클릭	주점·푸드트럭 등 부스 리스트를 드로어로 노출
드로어	높이 20%→80%로 끌어올리는 인터랙션

INTERACTION (INP)

1130 → 330 → 34ms

약 97% 단축

ROLE 프론트엔드 · 지도 페이지 · 성능 측정·개선

STACK React · TypeScript · Mapbox · framer-motion

PROBLEM

기능은 다 됐지만, 리스트를 여닫는 순간 애니메이션이 통째로 끊겼습니다.

- 1

여닫을 때 프레임 전부 드롭

리스트를 여닫는 애니메이션이 통째로 드랍됐습니다.
- 2

가장 많이 만지는 화면

끊김이 그대로 체감으로 이어졌습니다.
- 3

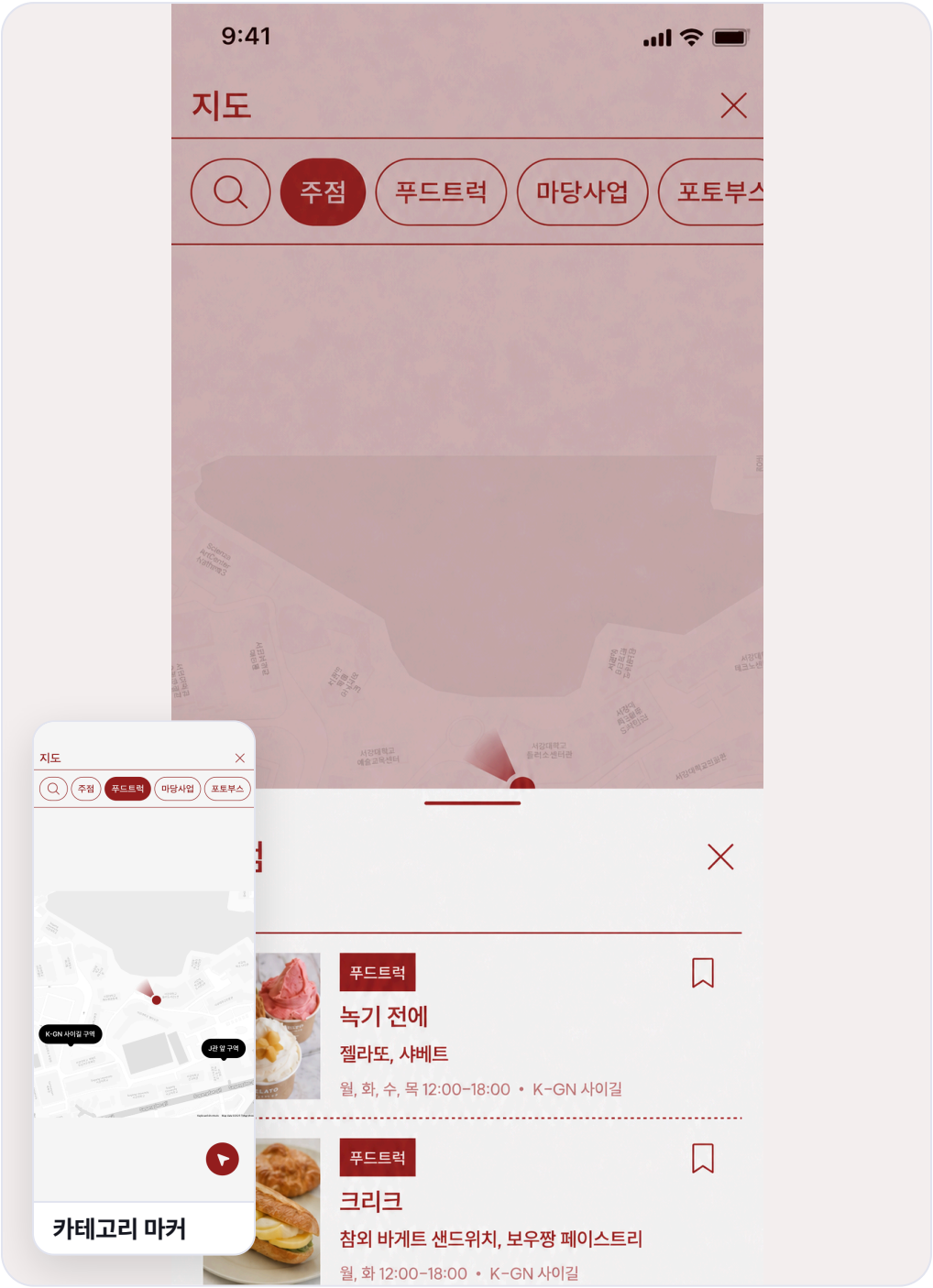
일반 지표로는 측정 불가

LCP·TBT가 안 잡혀 Lighthouse 실패.

Lighthouse 대신 성능탭·Clarity로 직접
측정해 병목을 좁혔습니다.

문제가 일어나는 실제 화면

마커 → 부스 리스트 드로어



마커 클릭 → 부스 리스트 드로어 · 여닫을 때 프레임 전부 드랍

— 병목 추적

가벼운 시도로 시작해, **측정으로 진짜 병목까지** 좁혀갔습니다

1 Lazy Load

효과 제한

이미지를 늦게 불러오면 되겠지

가설

이미지를 한 번에 로드하는 게 원인이라 보고, **가장 가벼운 lazy load**부터 시도했습니다.

리스트의 모든 썸네일 `<img>`

```
<img
  src={src}
  loading="lazy"
  decoding="async"
  fetchPriority="low"
/> // 펼치면 전부 한 번에 마운트
```

- **기대** — 화면 밖 이미지 요청을 미뤄 초기 부담 감소
- **실측** — 체감은 조금 부드러워졌으나 지표 개선은 제한적

원인 분리

lazy 는 **요청 시점** 힌트일 뿐, 펼치면 **항목 전체가 DOM에 마운트**되는 건 그대로였습니다.

2 IntersectionObserver

부분 개선

렌더 시점을 앱에서 직접 제어하자

시도

브라우저 판단 대신 **IntersectionObserver로 화면 근처 항목만** `<img>` 를 생성하도록 직접 제어했습니다.

LazyImage.tsx

```
const io = new IntersectionObserver((e) => {
  if (e[0].isIntersecting) {
    setInView(true);
    io.unobserve(e[0].target);
  }
}, { root, rootMargin, threshold });

{inView && !error && <img src={src} ... />}
```

- **rootMargin**으로 미리 로드해 깜빡임 방지
- 감지 즉시 `unobserve` 로 재관찰 차단
- 재사용 컴포넌트라 `memo` + `skeleton`

절반의 성공 · INP 1130 → 343MS

여는 건 부드러워졌지만 **닫을 땐 여전히 드랍**. 대상 수만으로는 부족해 **이미지 처리 비용**을 점검했습니다.

3 원본 해상도

진짜 원인

네트워크탭을 다시, 자세히

발견

이미지 **받아오는 시간이 유독 길어**, 네트워크탭에서 원본 크기를 확인했습니다.

6000×4000 약 2,400만 픽셀

- 표시 크기 **600×400** 대비 픽셀 약 **100배**
- 작게 보여도 **원본 전체를 디코딩 후 축소**

정리

업로드 시 AWS Lambda 자동 리사이즈를 측정 근거와 함께 백엔드에 제안했습니다.

병목의 위치

디코딩이 메인 스레드를 동기 점유해, 그 사이 닫기 프레임이 그려지지 못한 것이 직접 원인이었습니다.

— 개선과 결과

프론트와 백엔드 개선을 함께 적용해 근본 원인을 해결했습니다

HOW · 프론트 한계를 측정으로 넘어 백엔드와 함께

FRONTEND · 내가 한 것

1130 → 343 ms

처리 대상 수 줄이기

IntersectionObserver로 화면 근처 항목만 `<img>` 를 생성해 디코딩 수를 줄였습니다.

프론트만으로는 한계.

보이는 몇 장조차 6000×4000 원본이라 디코딩이 메인 스레드를 점유.

BACKEND · 측정 근거로 제안

343 → 34 ms

측정 근거로 협업 요청

측정 근거를 정리해, 업로드 시 **AWS Lambda** 자동 리사이즈로 표시 크기에 맞춰 제공하자고 제안했습니다.

RESULT

INP — 마커 클릭 후 응답까지

1130 → 34 ms

처리 대상 수(프론트) × 장당 비용(백엔드)을 함께 낮춰 약 97% 단축

• 애니메이션 프레임 복구

전부 드랍 → 0

길게 늘린 리스트를 닫아도 드랍 없이 매끄럽게 동작합니다.

• 디코딩 부하 제거

2,400만 → 24만 px

리사이즈로 장당 픽셀 약 1/100, 메인 스레드 점유가 사라졌습니다.

• 네트워크 부하 감소

원본 전송 ↓

화면 근처 이미지만 표시 크기 해상도로 받아 전송량이 줄었습니다.

• 사용자 체감 확인

MS Clarity

세션 녹화에서 반복(분노) 클릭 감소를 확인했습니다.

프론트로 못 메우는 부분을 측정 근거와 함께 넘겨, 한 사람의 최적화가 아닌 팀의 해결로 마무리했습니다.

"성능을 개선했다"가 아니라, 측정으로 병목을 좁히고 책임 위치에 맞게 협업해 해결한 과정이었습니다.

기능은 동작했지만, 동시에 들어오면 데이터가 어긋났습니다.

03 / 베틱기 기반 강의실 예약 시스템

Django · DRF · PostgreSQL · 베틱기로 강의실을 배정하는 예약 백엔드

요청을 하나씩 보내면 멀쩡한데, 같은 잔액·같은 슬롯에 요청이 겹치면 값이 어긋났습니다.
앱의 비관적 락과 DB의 부분 유니크 인덱스로 양쪽에서 막았습니다.

01

앱 레벨 — 비관적 락
베틱기 20건이 동시에 들어와도 잔액에는 1건만 반영되던 문제를 행 잠금으로 해결

반영 1 → **20** / 20건

02

DB 레벨 — 부분 유니크 인덱스
같은 방·날짜에 예약이 여러 건 잡히는 문제를 DB 제약으로 차단

활성 예약 15 → **1** 건

— 베팅 기반 강의실 예약 백엔드

동시에 들어오면
잔액과 예약이 어긋났습니다

BACKGROUND

마일리지를 베팅해 강의실을 배정받는 예약 서비스입니다. 잔액이 떨어지면 배정 결과도 틀어집니다.

돈이 움직이는 지점

베팅	잔액 차감 + 베팅 기록 + 거래내역 생성
취소	경매 중이면 참여자 전원에게 환불
예약	같은 방·날짜 슬롯을 한 팀이 점유

예약 상태 6단계

- PENDING

BIDS_OPEN

BIDS_CLOSED

CONFIRMED

REJECTED

CANCELLED

동시 요청 시 실측 (BEFORE)

차감 반영

1건치

베팅 20건

활성 예약

15건

맞는 값 1건

ROLE 백엔드 · 동시성 제어 · 테스트 설계

STACK Django · DRF · PostgreSQL 15 · pytest

PROBLEM

하나씩 보내면 전부 통과했지만, 요청이 겹치는 순간 값이 어긋났습니다.

- 1

차감이 통째로 사라짐

잔액

베팅 20건이 모두 생성됐는데 잔액에는 1건만 반영됐습니다. 나머지 19건의 차감이 사라졌습니다.
- 2

한 슬롯에 예약 15건

예약

모델에 유니크 제약이 없어 같은 방·날짜로 몇 건이든 만들어졌습니다.
- 3

이중 환불 가능

취소가 동시에 두 번 들어오면 둘 다 아직 진행 중으로 보고 환불을 두 번 실행합니다.
- 4

검증이 트랜잭션 밖

잔액을 확인하고 차감하기까지 틈이 있어, 확인한 잔액이 이미 지난 값이 됩니다.

고치기 전에 문제를 먼저 재현해, 검증이 헛돌지 않게 했습니다.

잔액이 어긋나는 메커니즘

Lost Update

	요청 A · 1,000 차감	요청 B · 2,000 차감	DB 잔액
t1	READ 10000	—	10000
t2	—	READ 10000	10000
t3	계산 → 9000	계산 → 8000	10000
t4	WRITE 9000	—	9000
t5	—	WRITE 8000	8000
맞는 값은 7,000. A의 차감 1,000이 덮여 사라졌습니다.			Lost Update

기존 베팅 처리 reserve/views.py

BEFORE

```
# 검증이 트랜잭션 밖 · request.user 는 지난 시점의 값
if request.user.mileage_balance < bet_amount:
    return Response(..., status=400)

with transaction.atomic():
    user.mileage_balance -= bet_amount
    user.save() # SET balance = <계산값>
```

그럼 예약 전에 SELECT로 확인하면?

- A

SELECT — 중복 없음 확인
- B

SELECT — 중복 없음 확인
- A

INSERT
- B

INSERT — 둘 다 들어감

확인한 시점과 저장하는 시점 사이의 틈, TOCTOU입니다. 그 사이에 다른 요청이 들어오면 앱 코드로는 걸러낼 수 없습니다. transaction.atomic 은 원자성(A)을 보장할 뿐, 경합은 격리(I)가 다루는 문제입니다.

— 해결 과정

앱과 DB 양쪽에 잠금을 걸어 두 문제를 없앴습니다

1

비관적 락

앱 레벨

잔액 행을 잠근 상태로 검증과 차감

선택

문제가 되는 건 같은 잔액 행을 동시에 수정할 때입니다.

`select_for_update` 로 그 행을 잠갔습니다.

베팅 처리 `reserve/views.py`

```
with transaction.atomic():
    # 행 잠금 후 잔액 재조회
    user = User.objects.select_for_update() \
        .get(pk=request.user.pk)

    if user.mileage_balance < bet_amount:
        return Response(..., 400)

    user.mileage_balance -= bet_amount
    user.save()
```

t1	FOR UPDATE 획득	—
t2	검증 · 차감	대기(blocked)
t3	COMMIT · 락 해제	9000 읽음

B는 A의 커밋 이후에 차감된 잔액을 읽습니다.

취소·환불
예약 행을 잠근 뒤 상태를 확인해 환불이 중복 지급되지 않도록 했습니다.

2

부분 유니크 인덱스

DB 레벨

DB 제약으로 중복 예약 차단

선택

앱에서 미리 조회해 확인하는 방식은 TOCTOU에 취약합니다.

저장 시점에 검사되도록 DB 제약으로 옮겼습니다.

예약 모델 `reserve/models.py`

```
UniqueConstraint(
    fields=('room', 'reservation_date'),
    # 취소(5)·거절(2) 제외 — 재예약 허용
    condition=~models.Q(status__in=(2, 5)),
    name='unique_active_reservation...',
)
```

- 조건 없이 걸면 취소한 슬롯을 재예약할 수 없습니다
- `IntegrityError` 는 **409 Conflict**로 변환해 응답합니다
- 충돌 시 트랜잭션이 중단되므로 `atomic` 으로 롤백합니다

상태 전이

기능마다 흩어져 있던 `if` 문을 전이 테이블로 모았습니다. 종료 상태는 다음 전이를 비워 뒀습니다.

제약의 적용 범위
다른 API, 배치, 직접 실행한 SQL에도 같은 규칙이 적용됩니다.

3

검증

적용 후

두 문제 모두 재현되지 않습니다

결과

앞에서 실측한 두 증상이 그대로 사라졌습니다. 같은 조건으로 다시 요청해도 값이 어긋나지 않습니다.

잔액 차감

동시 20건

적용 전

20건 중 1건만 반영 · 오차 19,000

적용 후

20건 전부 반영 · 오차 0

차감이 사라지지 않고 전부 남습니다.

중복 예약

동시 15건

적용 전

활성 예약 15건 · double booking

적용 후

활성 예약 1건 · 나머지 409

한 슬롯에 한 팀만 남습니다.

- `TransactionTestCase` — 실제 커밋이 있어야 충돌이 재현됩니다
- `threading.Barrier` 로 요청 시점을 맞춰 동시에 호출합니다

회귀 방지
같은 조건을 테스트로 남겨 이후 변경에서도 계속 확인됩니다.

— 결과와 회고

동시 요청에서도 **정합성이 유지되는** 구조가 됐습니다

RESULT · 동일 조건 재측정

잔액 차감 반영 — 동시 20건 베팅

4 → 20 / 20건

생성된 베팅이 **전부** 잔액에 반영됩니다.
요청이 몇 건이든 **항상** 일치합니다.

활성 예약 — 동시 15건 요청

15 → 1 건

1건만 **201**, 나머지 14건은 **409**로 거절됩니다.
취소한 슬롯은 **다시 예약**할 수 있습니다.

• 이중 환불 차단

락 + 상태 재확인

취소가 겹치면 두 번째는
이미 취소됨으로 거절됩니다.

• 불법 전이 차단

전이 테이블

취소된 예약을
다시 **확정**할 수 없습니다.

측정이 성립하는 조건

DBPostgreSQL — **행 단위 락**이 동작해야 측정이 성립합니다

테스트TransactionTestCase — 실제로 커밋해야 경합이 재현됩니다

동시성threading.Barrier로 **출발 시점을 맞춰** 동시에 요청합니다

WHY · 세 가지 대안 중에서

왜 비관적 락이었나

아래 세 방식 모두 Lost Update는 막습니다. 처리량을 조금 내주더라도 **재시도 없이 정확하게** 처리하는 쪽을 골랐습니다.

방식	성격	
비관적 락 select_for_update	경합이 한 행에 몰릴 때 유리합니다. 락 대기로 차례가 정해져 재시도 로직이 필요 없습니다.	채택
원자적 연산 F()	가장 가볍고 빠르지만, 검증 조건이 복잡하면 함께 쓰기 어렵습니다.	대안
낙관적 락 version	충돌이 드물 때 유리합니다. 경합이 잦으면 재시도가 몰려 오히려 느려집니다.	대안

| **정답이 하나가 아니라, 상황에 맞는 선택이 있었습니다.** 경합이 물리는 구조라 **처리량** 대신 **정확성**을 택했습니다.

THANK YOU

읽어주셔서 감사합니다

궁금한 점이나 함께 이야기하고 싶은 부분이 있다면
언제든 편하게 연락 주세요.

EMAIL

shu07002@gmail.com

PHONE

+82 10-3997-5917

GITHUB

github.com/shu07002

BLOG

shu07002.tistory.com